

CE27 Softwaretechnik

Übung

Prof.Dr. Thomas Baar

thomas.baar@htw-berlin.de



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

Sommersemester 2026

Heutige Themen

- Vergleich Hausaufgaben
- Arbeit am Projekt

Story – Beispiel AbhebenGeld

Erklären Sie, warum die untere UC-Beschreibung mangelhaft ist. Welche wichtigen Informationen fehlen? Wie würden Sie die UC Beschreibung umformulieren?

AbhebenGeld (Happy Day Szenario)

- 1) Das Use Case startet, sobald der Kunde die Bankkarte in den Automaten einsteckt.
- 2) Das System liest die Bankkarte und fordert den Kunden zur Eingabe der Personal Identification Number (PIN) auf.
- 3) Das System bietet ein Auswahlmenu an.
- 4) Der Kunde wählt das Abheben von Geld.
- 5) Das System fragt den abzuhebenden Betrag nach und der Kunde sollte den Betrag eingeben.
- 6) Geldnoten in Höhe des eingegebenen Betrags sowie die Karte werden ausgegeben.
- 7) Der Kunde wird das Geld und die Karte nehmen.
- 8) Das Use Case endet erfolgreich.

Story - Beispiel AbhebenGeld

Inhaltliche Fehler:

- Keine PIN-Eingabe durch den Kunden
- Keine Validierung der Kunden-Eingaben (PIN, Karte)

Sprachliche Fehler:

- Verwendung von Modalverben ("sollte" in 5)), Passiv ("werden" in 6)), und Futur ("wird" in 7))
- Keine Satzstruktur Subjekt-Prädikat-Objekt in 6) (resultierend aus Benutzung Passivkonstruktion)

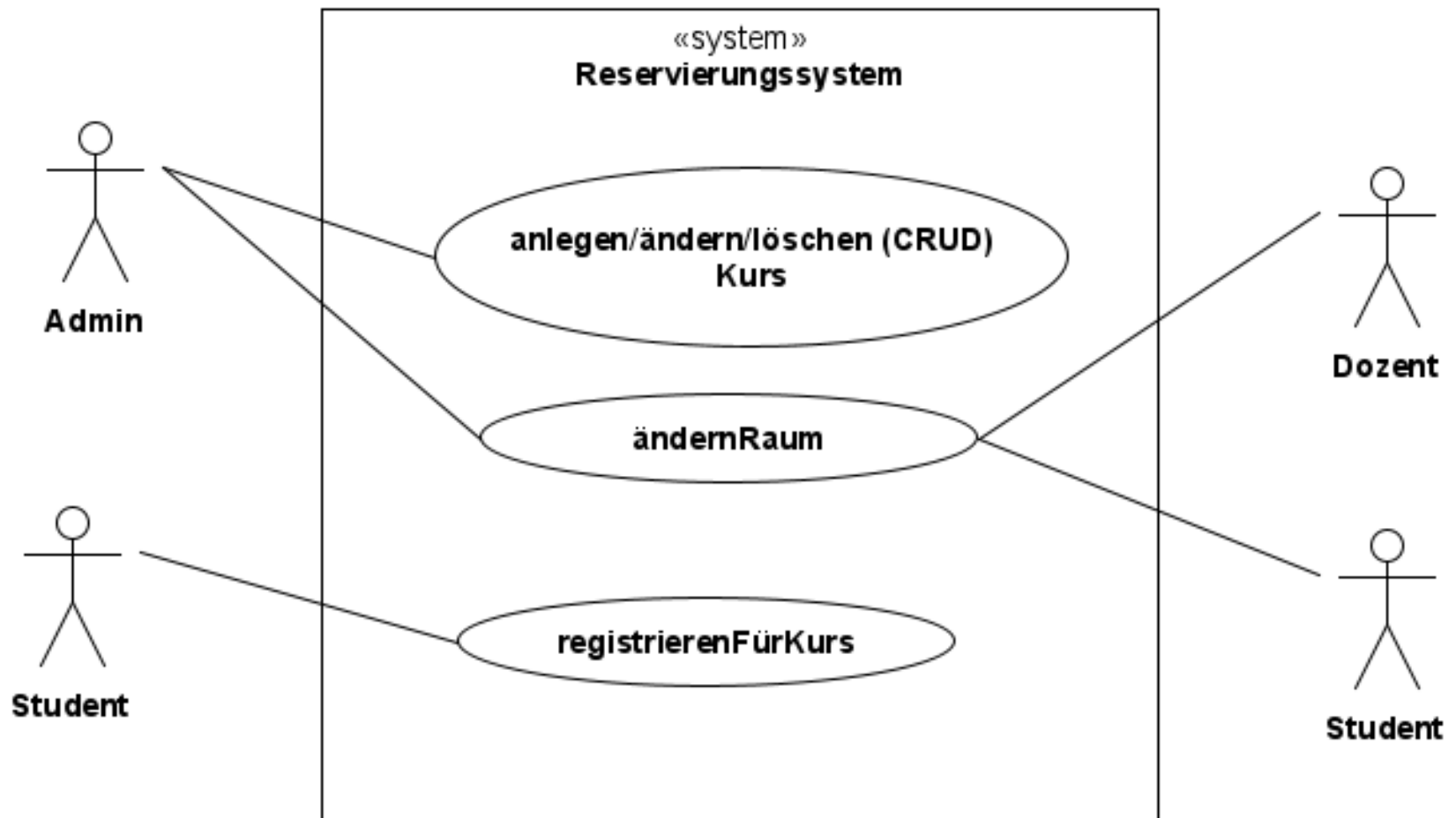
Sonstige Fehler:

- Zwei Aktionen mit unterschiedlichen Akteuren in einem Schritt (5))

Registrierungssystem

- Finden Sie Aktoren und Ziele in der folgenden informellen Beschreibung:
 - *Eine Universität möchte ein Registrierungssystem einführen, mit dem sich Studierende für ihre Kurse registrieren können. Die Liste der aktuellen Kurse kann nur von autorisierten Personen geändert werden.*
- Erarbeiten Sie eine textuelle Beschreibung für das Use Case *ÄndernKursRaum*. Dieses Use Case soll das Verlegen eines Kurses in einen anderen Raum ermöglichen. Beachten Sie, dass es bei der Verlegung zu keinen Konflikten mit anderen Kursen kommen darf (zu jeder Zeit darf in einem Raum immer nur ein Kurs stattfinden).

Registrierungssystem



Registrierungssystem

Use Case: ändernRaum

Kurzbeschreibung : Der Admin möchte den Raum ändern, in dem ein Kurs gegeben wird.

Primärer Akteur: Admin

Vorbedingung: Der Admin hat sich im System authentifiziert.

Nachbedingung: Es gibt keine zwei Kurse zur selben Zeit im selben Raum.

Registrierungssystem

Erfolgsszenario:

- 1) Der Admin wählt die Funktion "Raumänderung".
- 2) Das System listet alle Kurse auf.
- 3) Der Admin wählt den gewünschten Kurs aus.
- 4) Der Admin gibt den neuen Raum ein.
- 5) Das System validiert die Änderung und benachrichtigt alle involvierten Studenten und Dozenten.
- 6) Use Case endet erfolgreich.

Registrierungssystem

Erweiterungen:

2||a. Die Kursliste **ist** leer:

2||a.1. Das System **informiert** den Admin.

2||a.2. Use Case endet erfolglos.

(3-4)a. Das System **überschreitet** beim Warten auf die Eingabe ein Zeitlimit.

(3-4)a.1. Das System **löscht** die Authentifizierung des Admin.

(3-4)a.2. Use Case endet erfolglos.

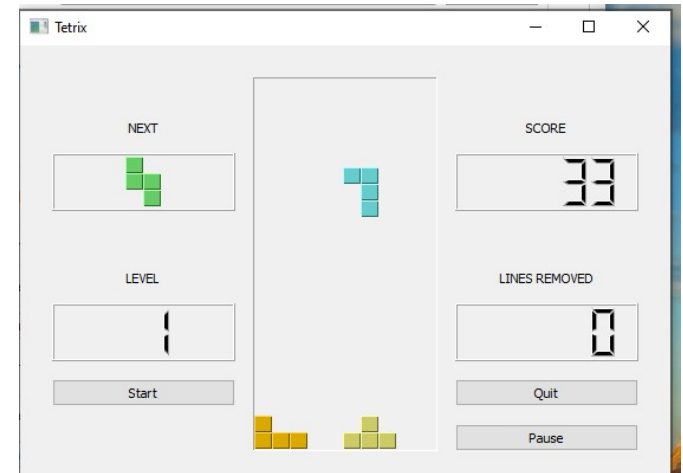
5a. Das System **stellt** Fehler bei der Validierung der Raumänderung **fest**. Mögliche Fehler sind inkorrekte Raumnummer oder Kollision mit anderem Kurs.

5a.1. Das System **informiert** den Admin über den Fehler.

5a.2 Das Use Case wird in Schritt 3 oder 4 fortgesetzt.

Beschreibung Tetris

In der Qt Distribution (auch in Community-Version) finden sich viele wundervolle Programmier-Beispiel, unter anderem das Projekt "Tetrix"



- Erstellen Sie eine textuelle UC-Beschreibung für den UseCase "Spielen", mit der die Regeln des Tetris-Spiels klar werden
 - Anmerkung: Für diese Aufgabe brauchen Sie nicht in den Code zu schauen
- Schauen Sie in den Implementierungscode und versuchen Sie, diesen Code zu "visualisieren". Welche Aspekte des Codes werden durch Ihre Visualisierung erfasst?

Beschreibung Tetris

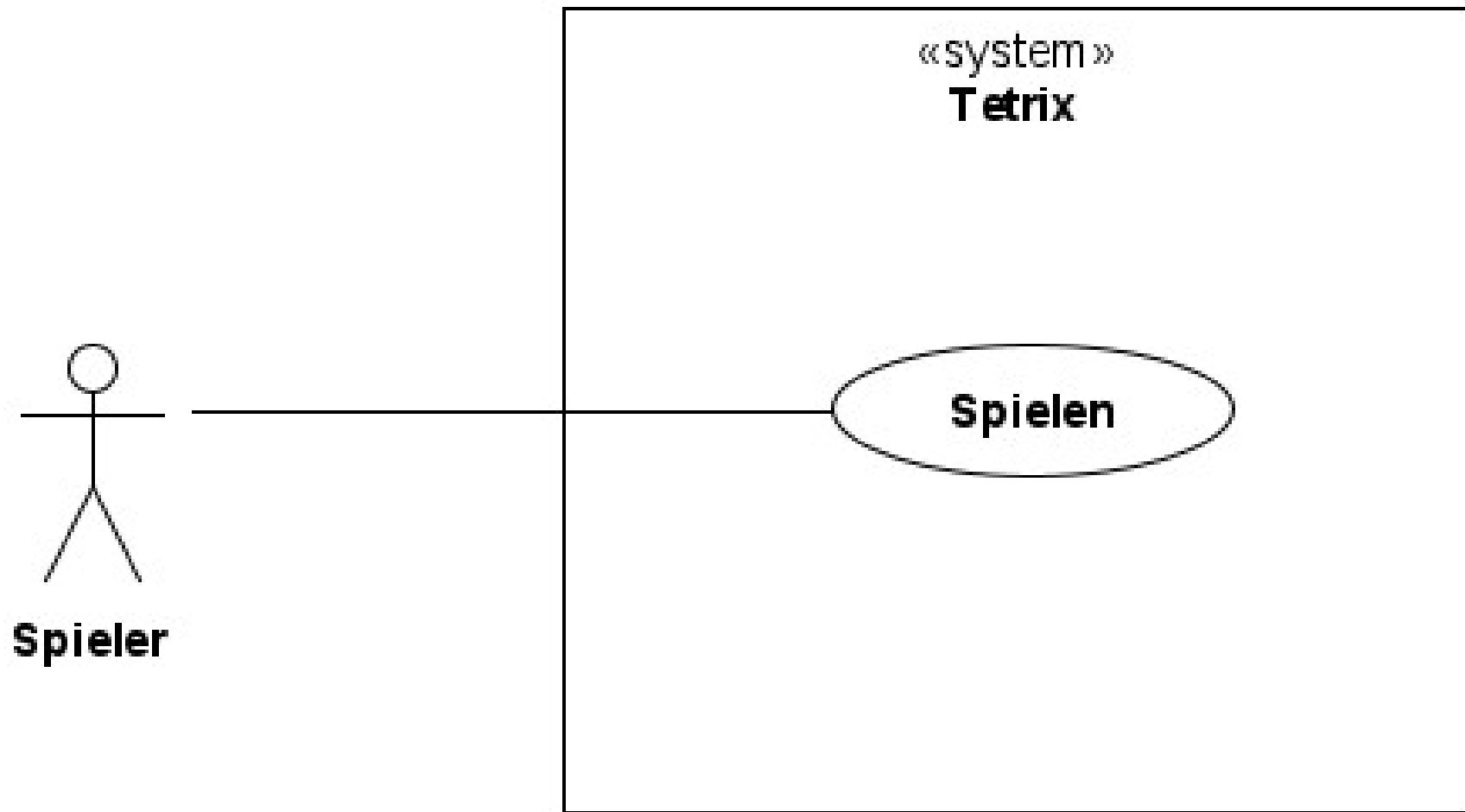
Glossar:

Item - Spielfigur, die auf dem Spielfeld bewegt wird.

Jede Spielfigur besteht aus vier kleinen Kästchen.

Wenn eine Zeile gelöscht wird, werden manchmal nicht alle Kästchen der Spielfigur gelöscht.

Beschreibung Tetris



Beschreibung Tetris

Use Case: Spielen

Kurzbeschreibung : Das Spiel besteht aus dem getakteten Fallen der aktuellen Spielfigur (*Item*) auf einem Spielfeld von oben nach unten. Der Spieler möchte durch geschickte Drehung und horizontale Verschiebung der Items erreichen, dass Zeilen des Spielfelds keine Leerstellen mehr haben. Die Bewegungsfreiheit des Items ist dabei durch bestehende Items auf dem Feld eingeschränkt. Wenn eine Zeile voll ist, wird sie aus dem Spielfeld entfernt und die verbleibenden Items rutschen eine Zeile nach unten. Das Spiel endet wenn ein Item sofort nach dem Hineinfallen in das Feld nicht mehr bewegt werden kann.

Beschreibung Tetris

Primärer Akteur: Spieler

Vorbedingung: Spiel wurde gestartet und es wird der Anfangsbildschirm gezeigt.

Nachbedingung: keine

Beschreibung Tetris

Erfolgsszenario:

- 1) Spieler **startet** ein neues Spiel.
- 2) System **fügt** ein neues Item dem Spielfeld am oberen Rand in einer bestimmten Spalte **hinzu**.
- 3) System **empfängt** Zeitsignal.
- 4) System **validiert**, dass Verschieben nach unten möglich ist.
- 5) System **vollführt** Bewegung nach unten.
- 6) UC **wird** in Schritt 3 **fortgesetzt**.

Beschreibung Tetris

Erweiterungen:

2a. System **stellt fest**, dass kein Platz für das neue Item vorhanden ist.

2a.1. System **informiert** den Spieler, dass das aktuelle Spiel beendet ist.

Beschreibung Tetris

Erweiterungen:

3a. Spieler **tätigt Eingabe** zum Bewegen (Drehen, Horizontale Verschiebung) des Items.

3a.1. System **validiert**, dass die Bewegung möglich ist

3a.2. System **führt** diese Bewegung **aus**.

3a.3. UC **wird** in Schritt 3 **fortgesetzt**.

3a.1a. System **findet heraus**, dass Bewegung nicht möglich ist.

3a.1a.1. UC **wird** in Schritt 3 **fortgesetzt**.

Beschreibung Tetris

Erweiterungen:

4a. System **findet heraus**, dass Bewegung nach unten nicht möglich ist.

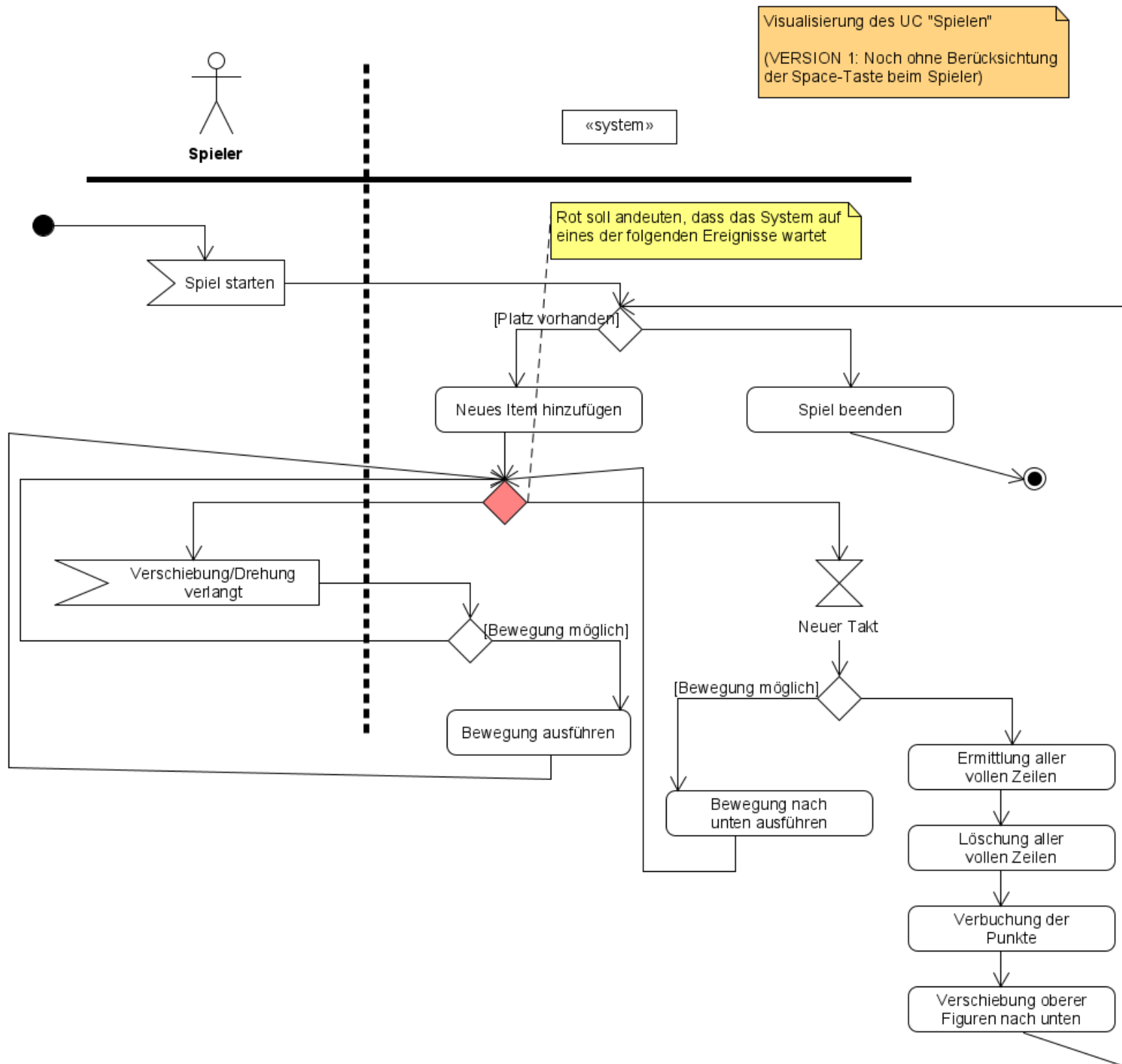
4a.1. System **ermittelt alle vollen Zeilen**, **löscht diese**, **erhöht** die Punkte des Spielers und **verschiebt** alle oberen Figuren entsprechend nach unten.

4a.2. UC **wird in Schritt 2 fortgesetzt**.

Beschreibung Tetris

Textuelle UC Beschreibung als Aktivitätsdiagramm:

- siehe nächste Folie



Beschreibung Tetris

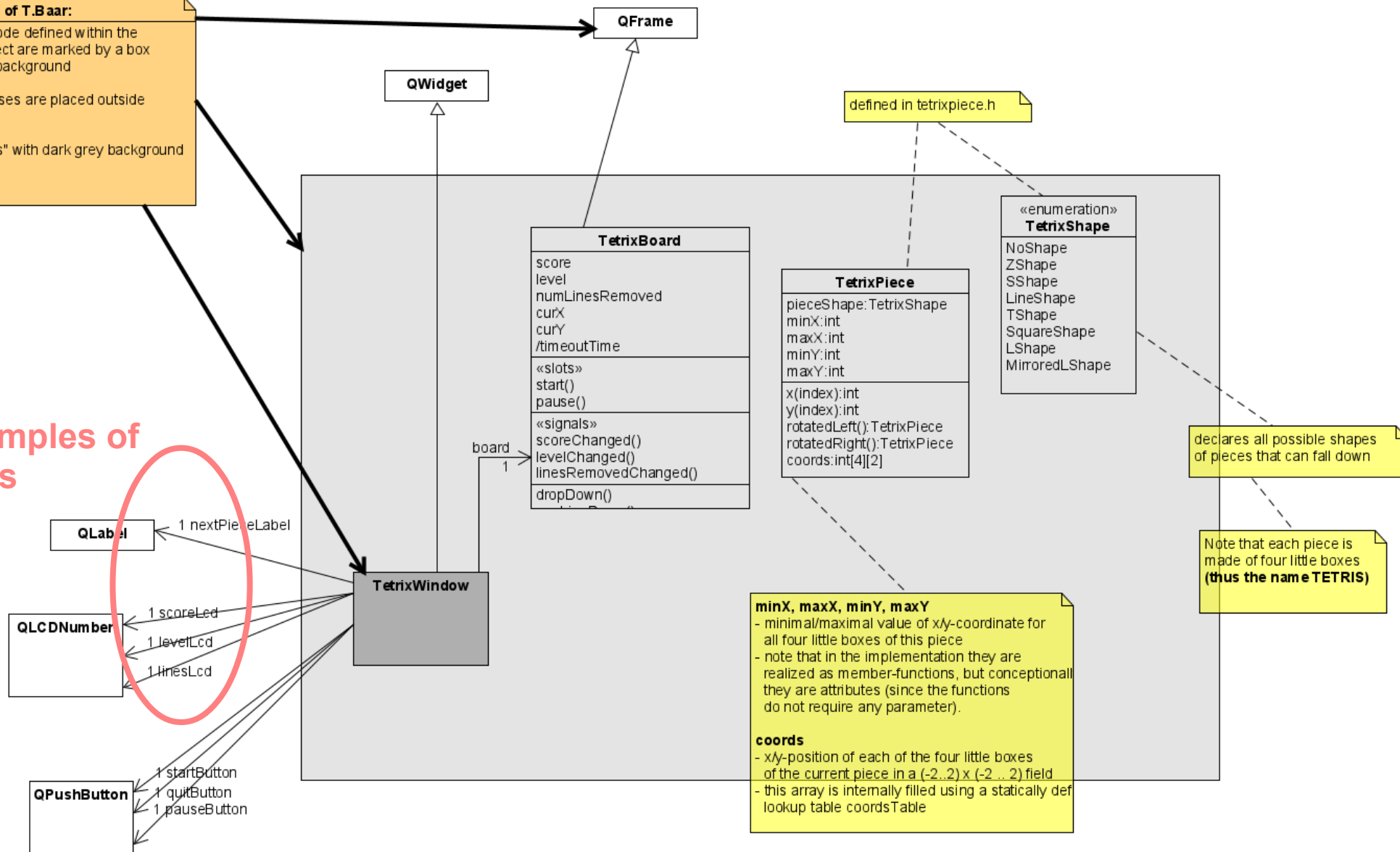
Detailliertes Klassendiagramm zur Visualisierung des Codes (hier: könnte auch als sehr feingranulares Domänenmodell dienen):

- siehe nächste Folie

Convention of T.Baar:

- Classes/Code defined within the current project are marked by a box with a gray background
- library classes are placed outside the gray box
- "main class" with dark grey background

Examples of roles



tetrixwindow.h

```
51 #ifndef TETRIXWINDOW_H
52 #define TETRIXWINDOW_H
53
54 #include <QWidget>
55
56 QT_BEGIN_NAMESPACE
57 class QLCDNumber;
58 class QLabel;
59 class QPushButton;
60 QT_END_NAMESPACE
61 class TetrixBoard;
62
63 /*! [0]
64 class TetrixWindow : public QWidget
65 {
66     Q_OBJECT
67
68 public:
69     TetrixWindow(QWidget *parent = nullptr);
70
71 private:
72     QLabel *createLabel(const QString &text);
73
74     TetrixBoard *board;
75     QLabel *nextPieceLabel;
76     QLCDNumber *scoreLcd;
77     QLCDNumber *levelLcd;
78     QLCDNumber *linesLcd;
```

Roles become
Member variables

tetrixwindow.cpp

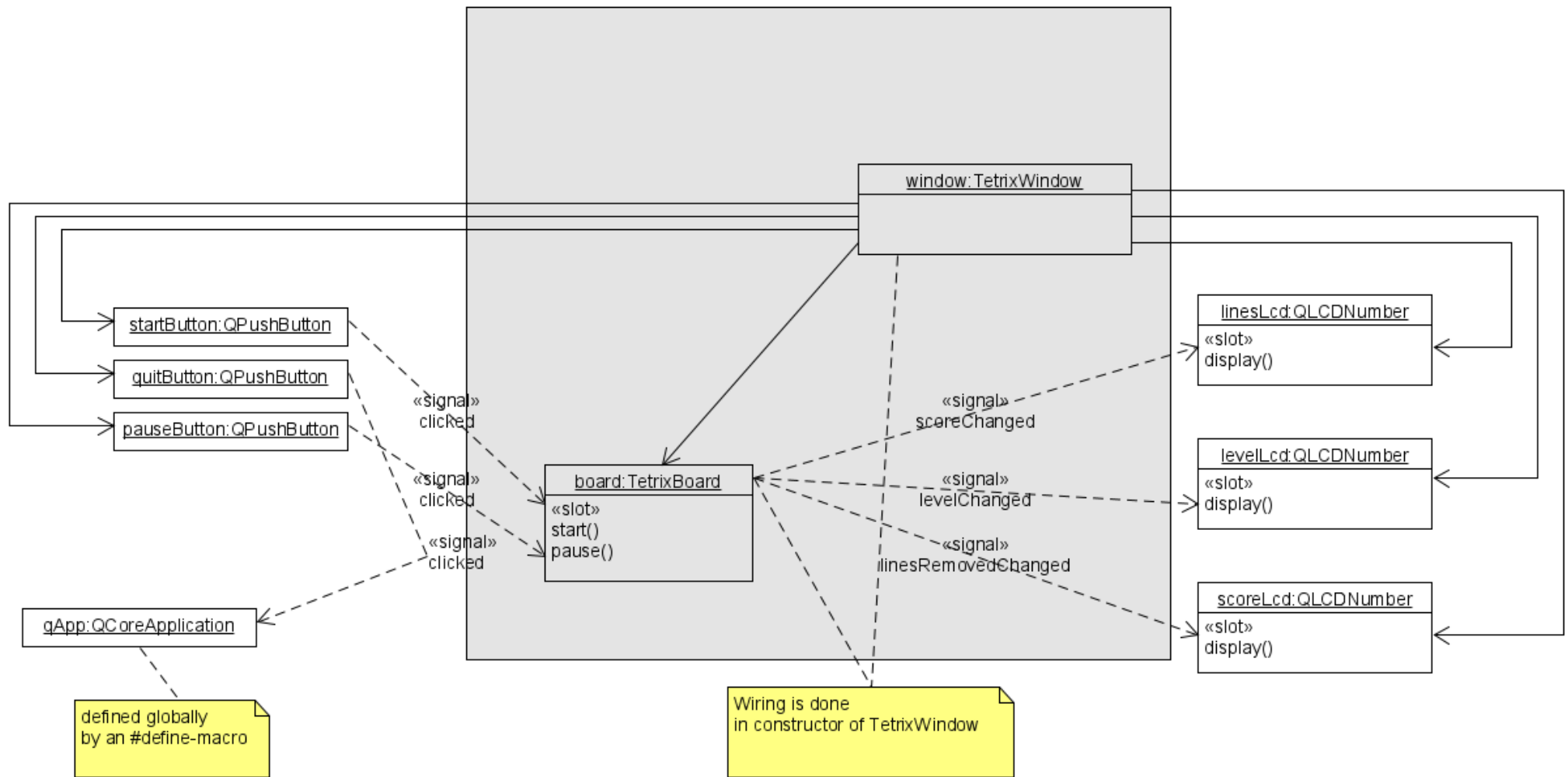
(Signallauf-Definition in Konstruktor)

```
88     connect(startButton, &QPushButton::clicked, board, &TetrixBoard::start);
89     //! [4] //! [5]
90     connect(quitButton, &QPushButton::clicked, qApp, &QCoreApplication::quit);
91     connect(pauseButton, &QPushButton::clicked, board, &TetrixBoard::pause);
92     #if __cplusplus >= 201402L
93     connect(board, &TetrixBoard::scoreChanged,
94             scoreLcd, qOverload<int>(&QLCDNumber::display));
95     connect(board, &TetrixBoard::levelChanged,
96             levelLcd, qOverload<int>(&QLCDNumber::display));
97     connect(board, &TetrixBoard::linesRemovedChanged,
98             linesLcd, qOverload<int>(&QLCDNumber::display));
99     #else
100     connect(board, &TetrixBoard::scoreChanged,
101            scoreLcd, QOverload<int>::of(&QLCDNumber::display));
102     connect(board, &TetrixBoard::levelChanged,
103            levelLcd, QOverload<int>::of(&QLCDNumber::display));
104     connect(board, &TetrixBoard::linesRemovedChanged,
105            linesLcd, QOverload<int>::of(&QLCDNumber::display));
106     #endif
107     //! [5]
108
109     //! [6]
110     QGridLayout *layout = new QGridLayout;
111     layout->addWidget(createLabel(tr("NEXT")), 0, 0);
112     layout->addWidget(nextPieceLabel, 1, 0);
113     layout->addWidget(createLabel(tr("LEVEL")), 2, 0);
```

Wiring senders
and receivers
of signals

tetrixwindow.cpp

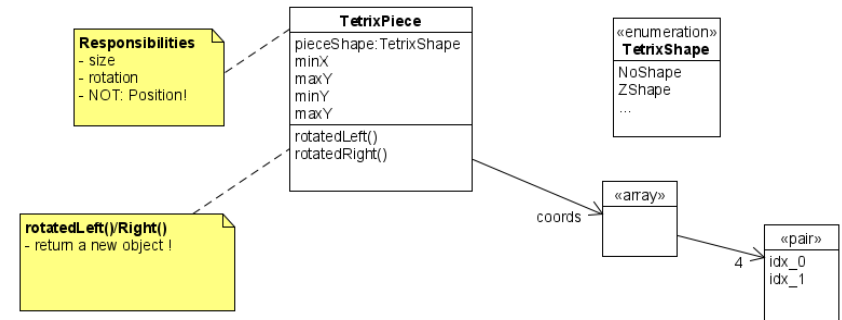
(Signallauf-Definition in Konstruktor)



Rotation of TetrixPiece

```
void TetrixPiece::setShape(TetrixShape shape)
{
    static constexpr int coordsTable[8][4][2] = {
        { { 0, 0 }, { 0, 0 }, { 0, 0 }, { 0, 0 } },
        { { 0, -1 }, { 0, 0 }, { -1, 0 }, { -1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 1, 0 }, { 1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 0, 1 }, { 0, 2 } },
        { { -1, 0 }, { 0, 0 }, { 1, 0 }, { 0, 1 } },
        { { 0, 0 }, { 1, 0 }, { 0, 1 }, { 1, 1 } },
        { { -1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } },
        { { 1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } }
    };

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 2; ++j)
            coords[i][j] = coordsTable[shape][i][j];
    }
    pieceShape = shape;
};
```



```
TetrixPiece TetrixPiece::rotatedLeft() const
{
    if (pieceShape == SquareShape)
        return *this;

    TetrixPiece result;
    result.pieceShape = pieceShape;
    for (int i = 0; i < 4; ++i) {
        result.setX(i, y(i));
        result.setY(i, -x(i));
    }
    //! [7]
    return result;
}
```

rotatedLeft()

```
void TetrixPiece::setShape(TetrixShape shape)
{
    static constexpr int coordsTable[8][4][2] = {
        { { 0, 0 }, { 0, 0 }, { 0, 0 }, { 0, 0 } },
        { { 0, -1 }, { 0, 0 }, { -1, 0 }, { -1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 1, 0 }, { 1, 1 } },
        { { 0, -1 }, { 0, 0 }, { 0, 1 }, { 0, 2 } },
        { { -1, 0 }, { 0, 0 }, { 1, 0 }, { 0, 1 } },
        { { 0, 0 }, { 1, 0 }, { 0, 1 }, { 1, 1 } },
        { { -1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } },
        { { 1, -1 }, { 0, -1 }, { 0, 0 }, { 0, 1 } }
    };

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 2; ++j)
            coords[i][j] = coordsTable[shape][i][j];
    }
    pieceShape = shape;
}
```

```
TetrixPiece TetrixPiece::rotatedLeft() const
{
    if (pieceShape == SquareShape)
        return *this;

    TetrixPiece result;
    result.pieceShape = pieceShape;
    for (int i = 0; i < 4; ++i) {
        result.setX(i, y(i));
        result.setY(i, -x(i));
    }
    //! [7]
    return result;
}
```

Newly created object
is returned

Example Z-Shape

tetrixboard.h

```
private:
```

```
enum { BoardWidth = 10, BoardHeight = 22 };
```

```
TetrixShape &shapeAt(int x, int y) { return board[(y * BoardWi
```

```
int timeoutTime() { return 1000 / (1 + level); }
```

```
int squareWidth() { return contentsRect().width() / BoardWidth
```

```
int squareHeight() { return contentsRect().height() / BoardHei
```

```
void clearBoard();
```

```
void dropDown();
```

```
void oneLineDown();
```

```
void pieceDropped(int dropHeight);
```

```
void removeFullLines();
```

```
void newPiece();
```

```
void showNextPiece();
```

```
bool tryMove(const TetrixPiece &newPiece, int newX, int newY);
```

```
void drawSquare(QPainter &painter, int x, int y, TetrixShape s
```

```
QBasicTimer timer;
```

tetrixboard.h

```
QBasicTimer timer;  
QPointer<QLabel> nextPieceLabel;  
bool isStarted;  
bool isPaused;  
bool isWaitingAfterLine;  
TetrixPiece curPiece;  
TetrixPiece nextPiece;  
int curX;  
int curY;  
int numLinesRemoved;  
int numPiecesDropped;  
int score;  
int level;  
TetrixShape board[BoardWidth * BoardHeight];
```

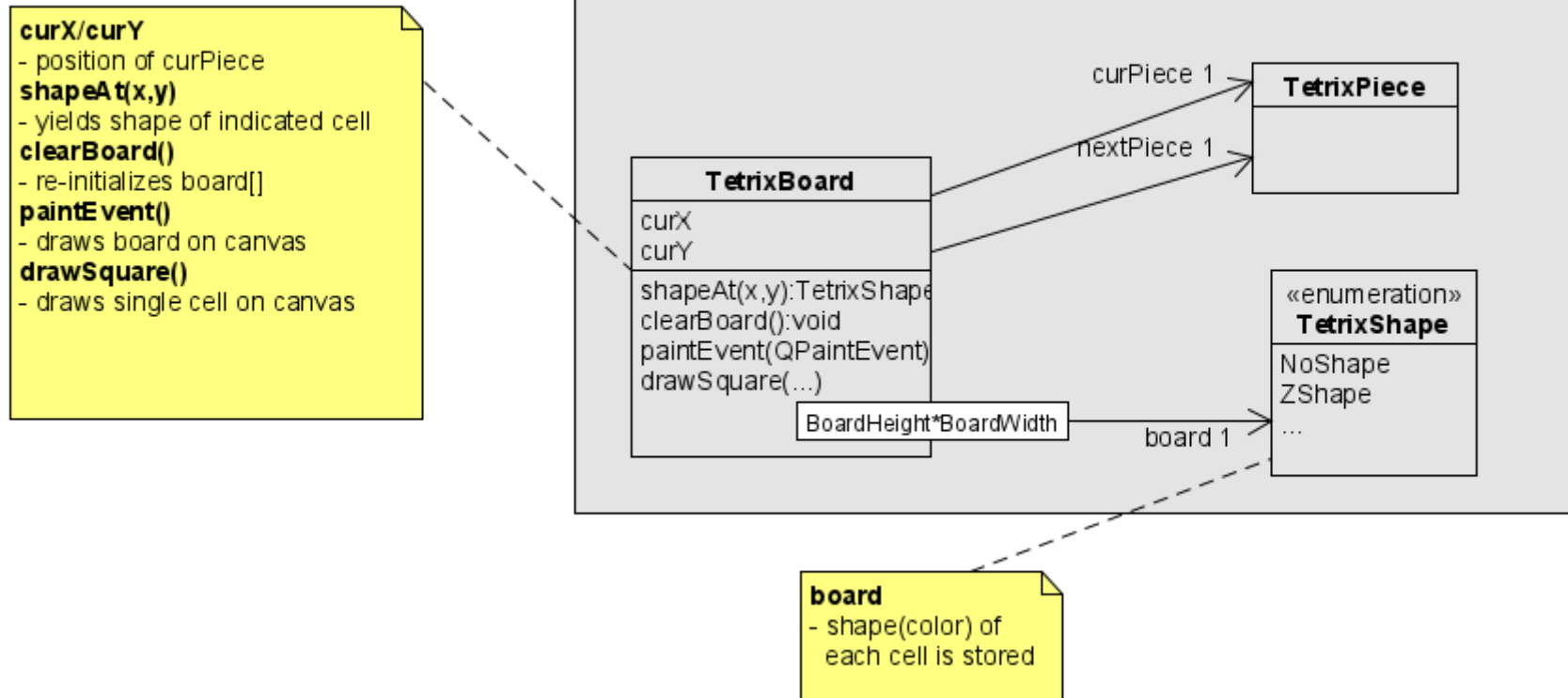
Rechteckiges Ausschneiden

Current and next
Piece are stored

Position of current
piece

Each cell of board
Is colored according
To assigned shape

tetrixboard.h



tetrixboard.cpp

Cleaning the board:

```
229 void TetrixBoard::clearBoard()
230 {
231     for (int i = 0; i < BoardHeight * BoardWidth; ++i)
232         board[i] = NoShape;
233 }
```

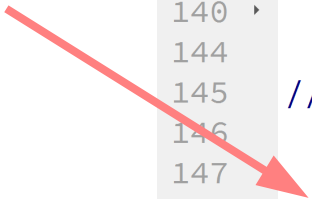
Accessing(reading) the board:

```
95 TetrixShape &shapeAt(int x, int y) { return board[(y * BoardWidth) + x]; }
```

tetrixboard.cpp

Drawing the board:

First step:
Draw all cells from
board[]



```
132 void TetrixBoard::paintEvent(QPaintEvent *event)
133 {
134     QFrame::paintEvent(event);
135
136     QPainter painter(this);
137     QRect rect = contentsRect();
138     //! [7]
139
140     if (isPaused) { ... }
141
142     //! [8]
143     int boardTop = rect.bottom() - BoardHeight*squareHeight();
144
145     for (int i = 0; i < BoardHeight; ++i) {
146         for (int j = 0; j < BoardWidth; ++j) {
147             TetrixShape shape = shapeAt(j, BoardHeight - i - 1);
148             if (shape != NoShape)
149                 drawSquare(painter, rect.left() + j * squareWidth(),
150                             boardTop + i * squareHeight(), shape);
151         }
152     }
153 }
154
```


tetrixboard.cpp

Drawing the board:

Second step:
Draw(overwrite) all cells from
curPiece



```
160 ▾ if (curPiece.shape() != NoShape) {  
161 ▾     for (int i = 0; i < 4; ++i) {  
162         int x = curX + curPiece.x(i);  
163         int y = curY - curPiece.y(i);  
164         drawSquare(painter, rect.left() + x * squareWidth(),  
165                     boardTop + (BoardHeight - y - 1) * squareHeight(),  
166                     curPiece.shape());  
167     }  
168     //! [10] //! [11]  
169 }
```

tetrixboard.cpp

Drawing a cell:

Knows the colors

Fills a rectangle
(given by pixel-position)

Draw four lines
(given by pixel-position)

```
397 void TetrixBoard::drawSquare(QPainter &painter, int x, int y, TetrixShape shape
398 {
399     static constexpr QRgb colorTable[8] = {
400         0x000000, 0xCC6666, 0x66CC66, 0x6666CC,
401         0xCCCC66, 0xCC66CC, 0x66CCCC, 0xDAAA00
402     };
403
404     QColor color = colorTable[int(shape)];
405     painter.fillRect(x + 1, y + 1, squareWidth() - 2, squareHeight() - 2,
406                     color);
407
408     painter.setPen(color.lighter());
409     painter.drawLine(x, y + squareHeight() - 1, x, y);
410     painter.drawLine(x, y, x + squareWidth() - 1, y);
411
412     painter.setPen(color.darker());
413     painter.drawLine(x + 1, y + squareHeight() - 1,
414                     x + squareWidth() - 1, y + squareHeight() - 1);
415     painter.drawLine(x + squareWidth() - 1, y + squareHeight() - 1,
416                     x + squareWidth() - 1, y + 1);
417 }
```